

## LICENCE 3 INFORMATIQUE

---

# Jeux de cartes, Blackjack

Méthode de Conception

---

*Auteurs :*

LIONEL NAHOU  
MOREL TALON  
ROBERTO HOUNGBO  
SÈNA GOUBALAN

*Chargé du cours :*

Yann MATHET

*Encadrant TP :*

Yann MATHET

# Table des matières

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>Introduction</b>                                             | <b>2</b>  |
| <b>1 État de l'art</b>                                          | <b>3</b>  |
| 1.1 Présentation du Jeu . . . . .                               | 3         |
| 1.2 Règles classiques du Jeu . . . . .                          | 3         |
| <b>2 Architecture du Projet</b>                                 | <b>5</b>  |
| 2.1 Structure générale du projet . . . . .                      | 5         |
| 2.2 Détails des Composants . . . . .                            | 6         |
| <b>3 Élément Technique</b>                                      | <b>8</b>  |
| 3.1 Description des classes . . . . .                           | 8         |
| 3.1.1 Architecture générale de la partie de Blackjack . . . . . | 8         |
| 3.1.2 Modélisation des joueurs et logique du croupier . . . . . | 9         |
| 3.1.3 Modèle des cartes et gestion du paquet . . . . .          | 9         |
| 3.2 Joueur IA . . . . .                                         | 10        |
| 3.2.1 Stratégie Simple . . . . .                                | 10        |
| 3.2.2 Stratégie Optimale . . . . .                              | 11        |
| <b>4 Expérimentation</b>                                        | <b>14</b> |
| 4.1 Lancement de l'application . . . . .                        | 14        |
| 4.2 L'interface en quelques images . . . . .                    | 15        |
| 4.2.1 Début de manche . . . . .                                 | 15        |
| 4.2.2 Partie en cours . . . . .                                 | 16        |
| 4.2.3 Mode avec robot . . . . .                                 | 16        |
| 4.2.4 Gestion du Split . . . . .                                | 17        |
| 4.2.5 Victoire du joueur . . . . .                              | 18        |
| <b>Conclusion</b>                                               | <b>19</b> |
| 4.3 Conclusion . . . . .                                        | 19        |
| 4.3.1 Bilan des travaux . . . . .                               | 19        |
| 4.3.2 Défis techniques surmontés . . . . .                      | 19        |
| 4.3.3 Améliorations envisageables . . . . .                     | 19        |

# Introduction

## Contexte

Le développement de jeux de cartes informatisés offre un cadre idéal pour mettre en pratique la programmation orientée objet, la modélisation et les principes d'architecture logicielle. Dans ce projet, il s'agit de concevoir une application autour du jeu de *Blackjack*, fondée sur une librairie générique destinée à créer et manipuler différents jeux de cartes.

Une caractéristique essentielle du travail réside dans la séparation nette entre la librairie « cartes », réutilisable pour d'autres projets, et l'implémentation du jeu de *Blackjack* qui en dépend. L'ensemble doit respecter le modèle MVC, garantissant une distinction claire entre logique métier, interaction et affichage.

## Objectifs

Le but de notre projet étant de réaliser une application permettant de jouer au jeu. Les objectifs techniques et pédagogiques clairement définis dans ce projet sont les suivantes :

### Objectifs techniques

1. Concevoir une librairie de cartes flexible et réutilisable, permettant de modéliser différents jeux de cartes et leurs manipulations.
2. Développer une version jouable du jeu de *Blackjack*, incluant au minimum les règles fondamentales, ainsi que des extensions possibles (split, assurance).
3. Mettre en œuvre une architecture MVC garantissant la séparation des responsabilités entre modèle, vue et contrôleur.
4. Implémenter des intelligences artificielles représentant des joueurs automatisés dotés de stratégies.

### Objectifs pédagogiques

1. Appliquer concrètement les principes de modélisation objet.
2. Concevoir une architecture modulaire, extensible et maintenable.
3. Développer une solution complète, de la conception à l'implémentation et aux tests.

## Organisation

Ce rapport est subdivisé en quatre chapitres. Le chapitre 1 traite de l'Etat de l'art. Dans ce dernier, nous avons parlé du jeu. Le chapitre 2 est consacré à l'architecture du projet. Les éléments techniques de notre application ont été présentés dans le chapitre 3. Dans le dernier chapitre (chapitre 4), nous avons montré comment utiliser l'application.

# Chapitre 1

## État de l’art

### 1.1 Présentation du Jeu

Le *Blackjack* est l’un des jeux de cartes les plus populaires dans les casinos et les jeux en ligne. Il oppose un ou plusieurs joueurs au croupier, chaque joueur ayant pour objectif d’obtenir une main dont la valeur se rapproche le plus possible de 21, sans jamais dépasser cette limite. Contrairement à de nombreux jeux de cartes, les joueurs ne s’affrontent pas entre eux : seul le duel joueur–croupier importe pour déterminer l’issue d’une manche.

Le *Blackjack* moderne provient du *Vingt-et-un*, un jeu apparu en Europe au XVII<sup>e</sup> siècle, puis standardisé avec l’essor des casinos américains. Sa popularité tient à la fois à sa simplicité apparente et à la profondeur stratégique qu’il offre : probabilités, prise de décision, gestion du risque et optimisation de la main sont au cœur de l’expérience de jeu.

Le jeu utilise en général un ou plusieurs paquets de 52 cartes classiques. Selon les variantes, certaines règles spécifiques peuvent être ajoutées (*split*, *double down*, *assurance*, etc.), ce qui enrichit le gameplay et influence

### 1.2 Règles classiques du Jeu

Les règles traditionnelles du *Blackjack* reposent sur des principes simples que l’on retrouve dans la plupart des variantes. Une partie se déroule en plusieurs étapes et obéit aux règles suivantes :

#### Valeur des cartes.

- Les cartes de 2 à 10 ont leur valeur nominale.
- Les figures (Valet, Dame, Roi) valent 10.
- L’As vaut 1 ou 11, selon ce qui avantage le joueur sans dépasser 21.

#### Déroulement d’une main.

1. Chaque joueur reçoit deux cartes, généralement visibles.
2. Le croupier reçoit également deux cartes, dont une seule est visible.
3. Le joueur peut alors choisir entre plusieurs actions :
  - **Hit** : tirer une nouvelle carte.
  - **Stand** : s’arrêter et conserver sa main.
  - **Double** : doubler la mise et tirer une seule carte supplémentaire (selon variantes).
  - **Split** : séparer deux cartes identiques pour jouer deux mains indépendantes (selon variantes).

4. Une main dépassant 21 est *bust* : le joueur perd immédiatement.
5. Une fois tous les joueurs arrêtés ou éliminés, le croupier joue sa main selon des règles fixes :
  - tirer tant que la valeur est inférieure ou égale à 16,
  - s'arrêter à partir de 17 (selon les casinos, le croupier peut être obligé de tirer sur un *soft 17*).

**Conditions de victoire.**

- Le joueur gagne si sa main est plus proche de 21 que celle du croupier, sans dépasser.
- Le joueur perd si sa main dépasse 21 ou si le croupier a une main strictement meilleure.
- Une égalité résulte généralement en *push* : la mise est simplement rendue au joueur.
- Un *Blackjack naturel* (As + 10/figure) sur les deux premières cartes bat toutes les mains non naturelles.

Ces règles forment la base du jeu et constituent le cadre dans lequel s'inscrit notre implémentation. Des mécanismes additionnels, tels que l'assurance ou les splits multiples, peuvent enrichir la logique, mais ne sont pas indispensables à une version minimale du projet.

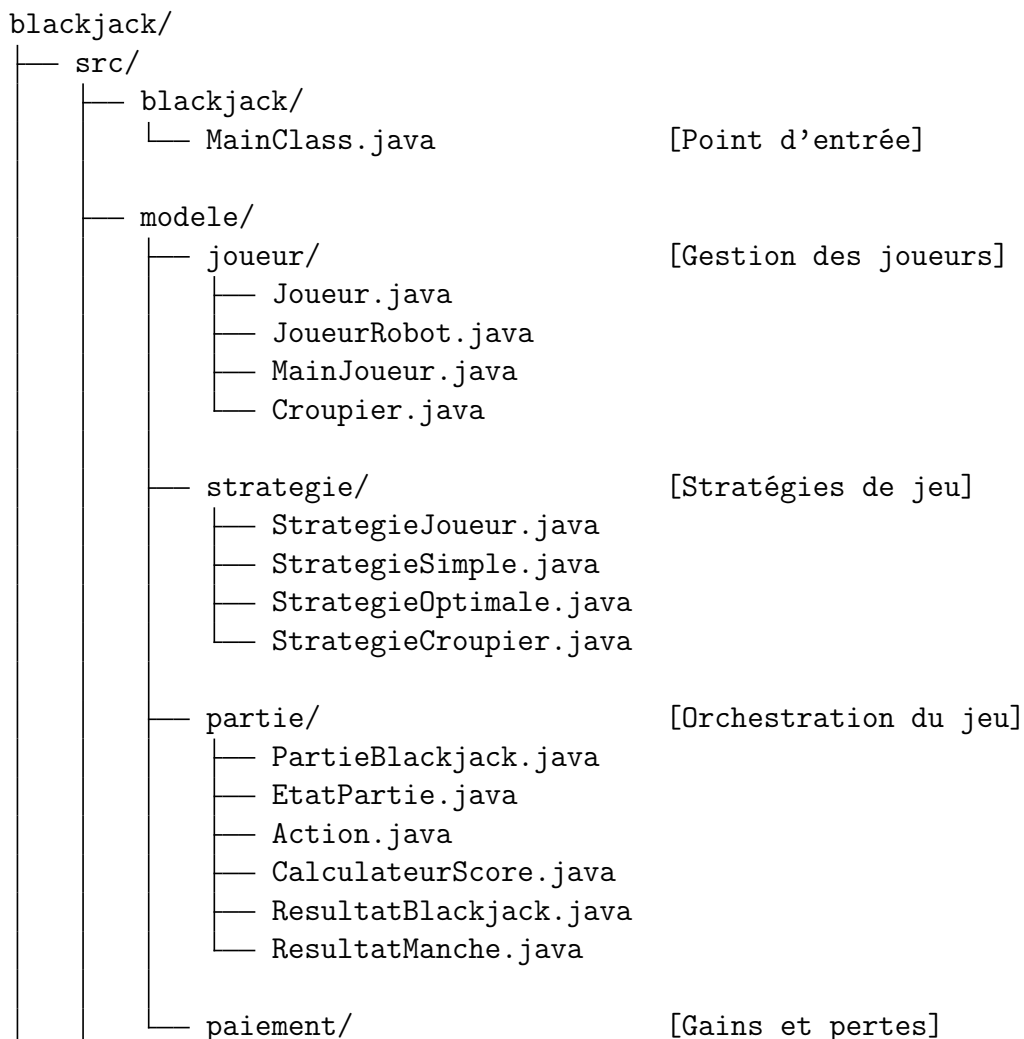
# Chapitre 2

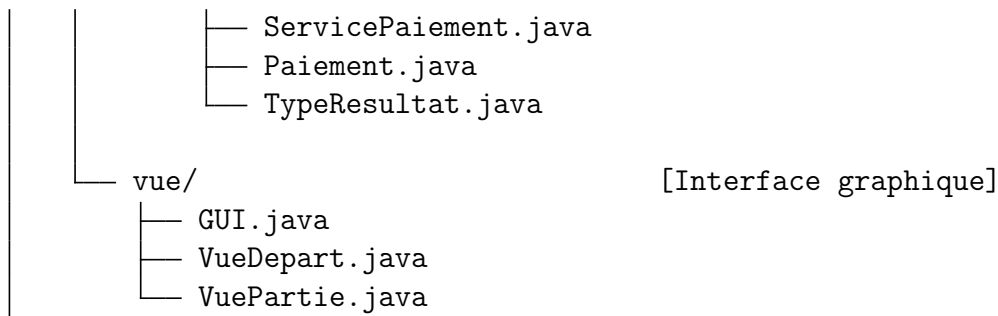
## Architecture du Projet

### 2.1 Structure générale du projet

La conception de l'application s'appuie sur une organisation modulaire visant à séparer clairement les responsabilités : logique métier, orchestration du jeu, stratégies des joueurs, gestion des paiements et interface graphique. Cette structuration favorise la lisibilité, la maintenabilité et l'évolution du projet, notamment grâce à la séparation nette entre le modèle, les stratégies et les vues. La structure suivante présente l'agencement global du projet.

#### Vue d'ensemble du projet





## 2.2 Détails des Composants

La structure du projet se décompose en plusieurs modules fonctionnels, chacun regroupant des classes coopérant autour d'une même responsabilité. Les principaux composants sont décrits ci-dessous.

### `modele.joueur`

Ce package rassemble l'ensemble des classes représentant les différents acteurs du jeu.

- **Joueur** : représente un joueur humain, incluant la gestion de sa banque, de ses mains et de ses mises.
- **JoueurRobot** : joueur automatisé utilisant une stratégie prédéfinie pour décider de ses actions.
- **MainJoueur** : modélise une main de jeu, comprenant les cartes associées ainsi que la mise correspondante.
- **Croupier** : représente le croupier, dont le comportement est défini par une stratégie fixe et une carte cachée.

### `modele.strategie`

Ce module implémente le patron de conception *Strategy*, permettant de définir et de substituer facilement différentes logiques de prise de décision.

- **StrategieJoueur** : interface commune à toutes les stratégies.
- **StrategieSimple** : stratégie basique tirant tant que la main est inférieure à 17, avec une gestion élémentaire du *split*.
- **StrategieOptimale** : implémentation d'une stratégie avancée fondée sur la *Basic Strategy*, optimisée statistiquement.
- **StrategieCroupier** : stratégie fixe imposée au croupier par les règles du Blackjack (tirage obligatoire en dessous de 17).

### `modele.partie`

Ce package regroupe les classes responsables de l'orchestration d'une partie et du déroulement des manches.

- **PartieBlackjack** : classe principale assurant la coordination des joueurs, la distribution des cartes et le déroulement complet d'une partie.
- **EtatPartie** : énumération décrivant les différents états successifs d'une partie (mise, distribution, actions, résolution, etc.).
- **Action** : énumération des actions possibles pour un joueur (TIRER, RESTER, DOUBLER, SÉPARER).
- **CalculateurScore** : utilitaire chargé du calcul de la valeur des mains, incluant la gestion des As *souples* et *durs*.

- **ResultatBlackjack** : représentation du résultat dans le cas d'un *blackjack naturel*.
- **ResultatManche** : encapsule le résultat complet d'une manche pour un joueur donné (victoire, défaite, égalité, blackjack).

#### `modele.paielement`

Ce module traite tous les aspects financiers liés aux gains et aux pertes des joueurs.

- **ServicePaielement** : assure le calcul des gains en fonction du résultat, applique les paiements et met à jour la banque des joueurs.
- **Paielement** : représente une transaction, comprenant le montant et la nature du paiement.
- **TypeResultat** : énumération des différents types de résultats (VICTOIRE, PERTE, PUSH, BLACKJACK).

#### `vue`

La partie graphique repose sur Swing et permet l'interaction avec l'utilisateur.

- **GUI** : fenêtre principale de l'application, assurant la gestion des vues.
- **VueDepart** : écran d'accueil proposant le lancement d'une partie.
- **VuePartie** : interface affichant la table de Blackjack et permettant de suivre le déroulement du jeu.



# Chapitre 3

## Élément Technique

### 3.1 Description des classes

#### 3.1.1 Architecture générale de la partie de Blackjack

Le premier diagramme UML présente l'architecture centrale de la logique de jeu. Il met en évidence la classe `PartieBlackjack`, qui coordonne l'ensemble du déroulement d'une manche, depuis la distribution initiale jusqu'à la détermination du résultat. Elle s'appuie sur un *sabot* (Paquet), un `Croupier` et une liste de `Joueur`, ce qui permet de gérer à la fois le joueur humain et les joueurs IA.

Le diagramme montre également les interactions avec les classes de gestion des résultats (`ResultatManche`), du paiement (`Paielement`, `ServicePaielement`) et des états de la partie (`EtatPartie`). On observe aussi la connexion avec la couche *Vue*, en particulier `VuePartie` et `VueDepart`, qui utilisent les services fournis par le modèle pour afficher le déroulement du jeu.

Ce premier diagramme permet donc de comprendre la structure globale du modèle MVC : le **modèle** repose sur `PartieBlackjack`, les **vues** sont assurées par Swing, et le rôle du **contrôleur** est intégré dans la logique des méthodes appelées par les interfaces graphiques.

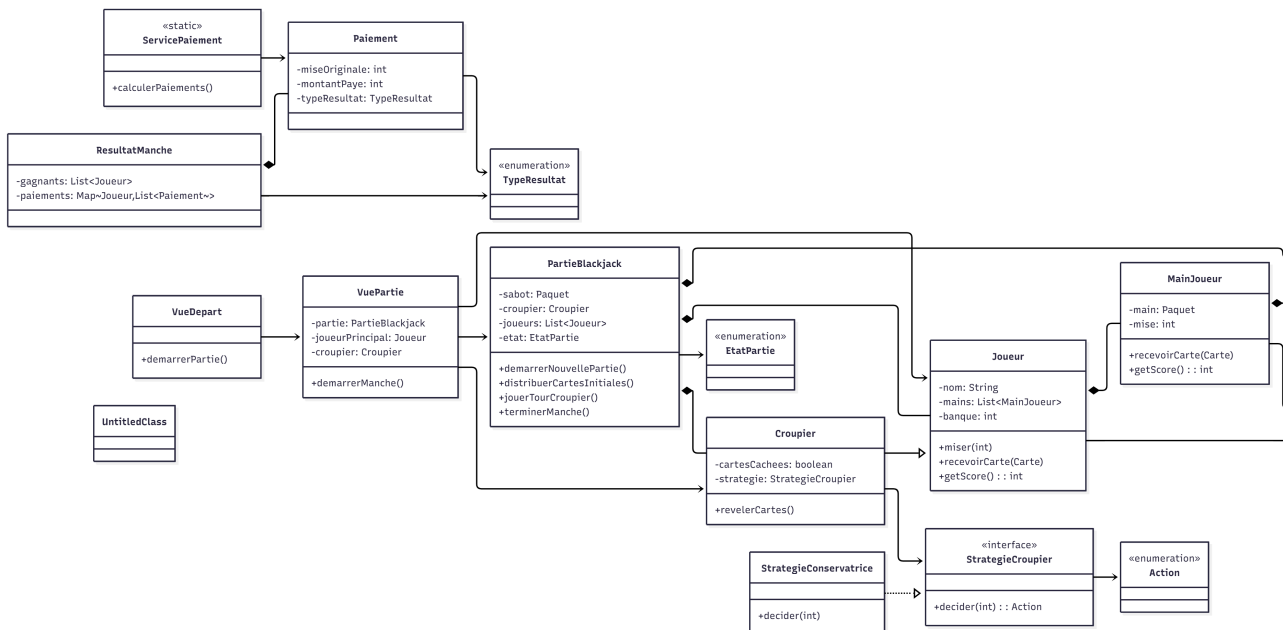


FIGURE 3.1 – Diagramme UML — Architecture générale de la partie de Blackjack

### 3.1.2 Modélisation des joueurs et logique du croupier

Le deuxième diagramme UML détaille les classes liées aux joueurs (**Joueur**, **JoueurRobot**, **Croupier**) ainsi que leur manière de représenter une main via la classe **MainJoueur**. Chaque joueur possède une banque, une liste de mains et des méthodes lui permettant de miser, recevoir des cartes ou calculer son score.

Le croupier hérite d'un comportement spécifique grâce à l'interface **StrategieCroupier**, qui impose une règle fondamentale du Blackjack : tirer tant que son total est inférieur à 17. Les joueurs IA, quant à eux, utilisent différentes implémentations de **StrategieJoueur**, comme la **StrategieSimple** ou la **StrategieOptimale**, leur permettant d'adapter leur style de jeu.

Ce diagramme illustre également l'interaction entre les mains du joueur et la classe **Paquet**, ainsi que l'utilisation de la classe utilitaire **CalculateurScore** pour déterminer la valeur d'une main, en tenant compte notamment du traitement particulier des As.

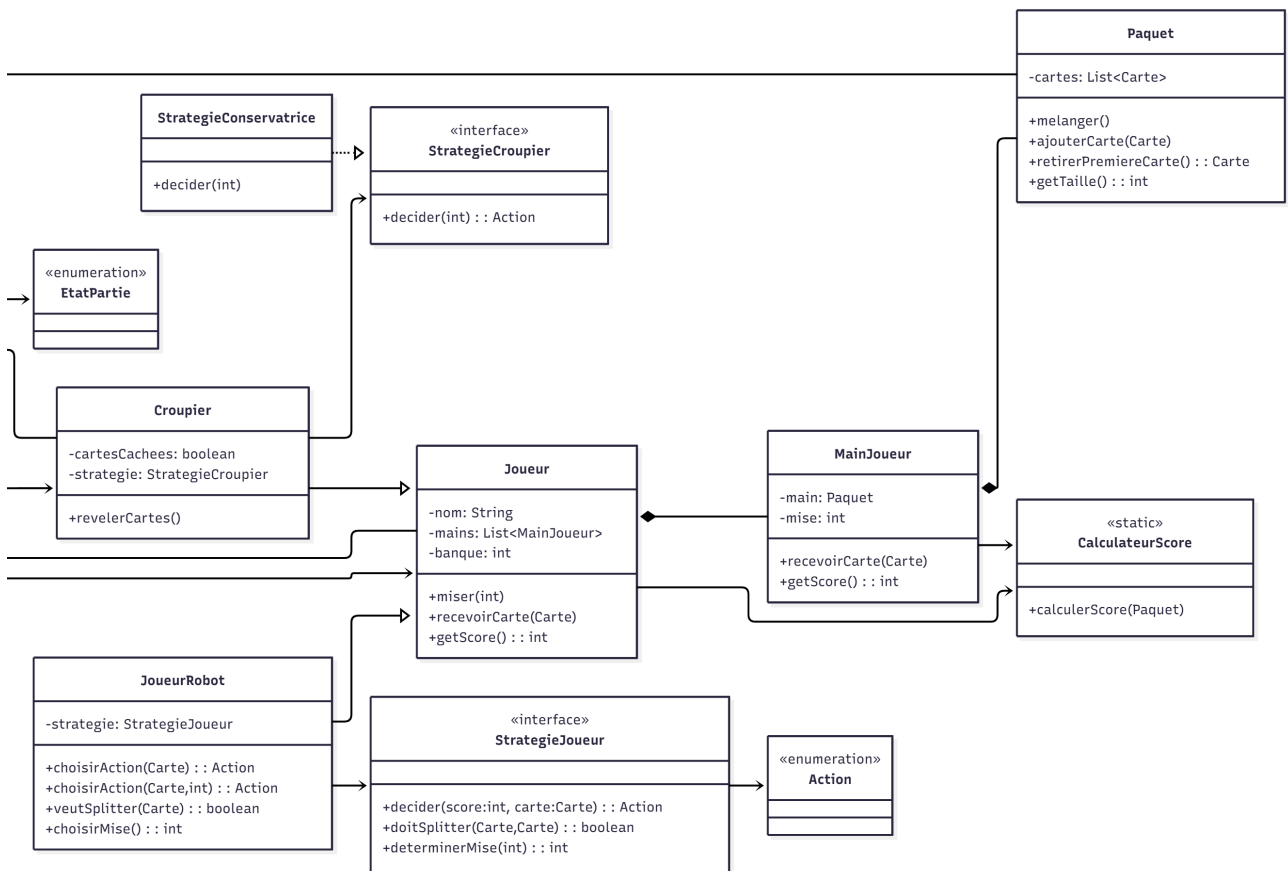


FIGURE 3.2 – Diagramme UML — Modélisation des joueurs et logique du croupier

### 3.1.3 Modèle des cartes et gestion du paquet

Le troisième diagramme UML se concentre sur la représentation du jeu de cartes, cœur de tout jeu de Blackjack. La classe **Carte** associe une **Couleur** et une **Hauteur**, représentées par deux énumérations distinctes. L'ensemble des cartes est contenu dans la classe **Paquet**, qui fournit les opérations fondamentales : mélanger, ajouter une carte, retirer la première carte et obtenir la taille restante du paquet.

La classe **MainJoueur** encapsule une main de cartes et la mise associée, ce qui permettra ensuite de calculer le résultat d'une manche. Le diagramme montre également l'utilisation de **CalculateurScore**, une classe utilitaire permettant de déterminer la valeur d'une main en appliquant les règles du Blackjack (As valant 1 ou 11, figures valant 10, etc.).

Ce diagramme met en lumière la modularité de la librairie « cartes », pensée pour être réutilisable dans d'autres jeux que le Blackjack.

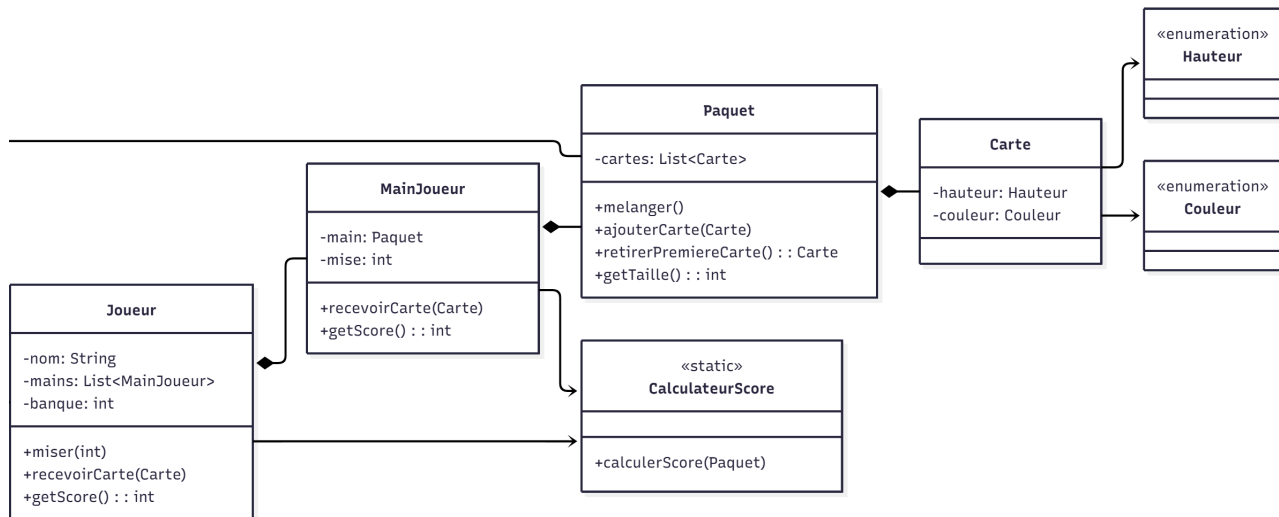


FIGURE 3.3 – Diagramme UML — Modèle des cartes et gestion du paquet

## 3.2 Joueur IA

Dans le cadre de ce projet, plusieurs comportements automatisés ont été implémentés afin de simuler des joueurs virtuels (robots). Ces intelligences artificielles reposent sur le patron de conception *Strategy*, permettant de substituer dynamiquement différentes logiques de décision pendant le jeu. Deux stratégies principales ont été développées : une stratégie simple, destinée à représenter un joueur peu expérimenté, et une stratégie optimale inspirée de la *Basic Strategy* professionnelle du Blackjack. [1].

### 3.2.1 Stratégie Simple

La stratégie simple constitue une approche minimaliste fondée sur un ensemble de règles élémentaires. Elle repose sur un seuil fixe de décision : le joueur tire une carte tant que son score est inférieur à 17, et reste sinon. Quelques règles additionnelles permettent d'améliorer légèrement le réalisme, notamment la possibilité de doubler certaines mains et de splitter les paires les plus critiques.

#### Principes de décision

- **Décision d'action** : tirer si le score est strictement inférieur à 17, rester sinon.
- **Double** : le joueur double sa mise lorsque son total vaut 9, 10 ou 11 et que la carte visible du croupier est favorable.
- **Split** : uniquement sur les paires d'As et de 8, conformément aux règles de base du Blackjack.

## Caractéristiques

- **Avantages** : stratégie très rapide, facile à interpréter et offrant un comportement cohérent.
- **Limites** : ce modèle ne tient pas compte des probabilités ; il n'est pas optimal et reste largement exploitable par les stratégies avancées.

## Algorithme de la Stratégie Simple

---

**Algorithme 1** : Décision d'action pour la stratégie simple

---

**Entrées** : scoreJoueur, carteVisibleCroupier

**Output** : Action à effectuer

```
1 si scoreJoueur < 17 alors
2 | retourner TIRER
3 sinon
4 | retourner RESTER
5 fin
```

---

---

**Algorithme 2** : Détermination de la mise pour la stratégie simple

---

**Entrées** : banque

**Output** : mise

```
1 mise ← 0.1 × banque
2 si mise < 10 alors
3 | mise ← min(10, banque)
4 sinon
5 | mise ← mise
6 fin
7 retourner mise
```

---

---

**Algorithme 3** : Décision de Split pour la stratégie simple

---

**Entrées** : cartePaire, carteVisibleCroupier

**Output** : booléen indiquant si Split

```
1 hauteur ← cartePaire.hauteur
2 si hauteur = AS ou hauteur = 8 alors
3 | retourner vrai
4 sinon
5 | retourner faux
6 fin
```

---

### 3.2.2 Stratégie Optimale

La stratégie optimale s'appuie sur la *Basic Strategy*, une approche mathématiquement établie qui minimise l'avantage du croupier. Cette stratégie analyse la main du joueur en fonction de la carte visible du croupier et applique des décisions optimisées : tirer, rester, doubler ou splitter selon les probabilités de gain.

Deux volets principaux composent cette stratégie : les *hard totals* (mains sans As compté comme 11) et la gestion des paires.

#### Hard Totals

- 17–20 : toujours rester.
- 13–16 : rester contre un croupier faible (2–6), tirer sinon.

- 12 : rester contre 4–6, tirer contre 2–3 et 7–As.
- 11 : toujours doubler.
- 10 : doubler contre 2–9, tirer contre 10–As.
- 9 : doubler contre 3–6, tirer sinon.
- 5–8 : toujours tirer.

### Split (paires)

- A,A et 8,8 : toujours séparer.
- 10,10 : ne jamais splitter.
- 9,9 : splitter contre 2–9 sauf 7.
- 7,7 : splitter contre 2–7.
- 6,6 : splitter contre 2–6.
- 5,5 : jamais splitter (traiter comme 10).
- 4,4 : splitter contre 5–6.
- 3,3 et 2,2 : splitter contre 2–7.

### Caractéristiques

- **Avantage** : réduit mathématiquement l’avantage de la maison et offre une performance supérieure à toutes les stratégies basiques.
- **Limites** : nécessite davantage de logique et de calculs ; ne gère pas encore complètement les *soft totals* (mains avec As compté comme 11) dans cette version du projet.

### Algorithme de la Stratégie Optimale (Hard Totals)

---

#### Algorithme 4 : Décision d’action pour une main “hard”

---

**Entrées** : scoreJoueur, valeurCroupier

**Output** : Action

```

1 si scoreJoueur ≥ 17 alors
2   retourner RESTER
3 si 13 ≤ scoreJoueur ≤ 16 alors
4   si 2 ≤ valeurCroupier ≤ 6 alors
5     retourner RESTER
6   retourner TIRER
7 si scoreJoueur = 12 alors
8   si 4 ≤ valeurCroupier ≤ 6 alors
9     retourner RESTER
10  retourner TIRER
11 si scoreJoueur = 11 alors
12   retourner DOUBLER
13 si scoreJoueur = 10 alors
14   si 2 ≤ valeurCroupier ≤ 9 alors
15     retourner DOUBLER
16   retourner TIRER
17 si scoreJoueur = 9 alors
18   si 3 ≤ valeurCroupier ≤ 6 alors
19     retourner DOUBLER
20   retourner TIRER
21 retourner TIRER

```

---

---

**Algorithme 5 : Détermination de la mise pour la stratégie optimale**

---

**Entrées :** banque**Output :** mise

```
1 mise  $\leftarrow$  0.15  $\times$  banque
2 si mise < 10 alors
3   | mise  $\leftarrow$  min(10, banque)
4 miseMax  $\leftarrow$  0.25  $\times$  banque
5 si mise > miseMax alors
6   | mise  $\leftarrow$  miseMax
7 retourner mise
```

---

---

**Algorithme 6 : Décision de Split pour la stratégie optimale**

---

**Entrées :** cartePaire, valeurCroupier**Output :** booléen

```
1 hauteur  $\leftarrow$  cartePaire.hauteur
2 hauteur cas où AS faire
3   | retourner vrai
4 cas où 10 faire
5   | retourner faux
6 cas où 9 faire
7   | si valeurCroupier = 7 ou valeurCroupier  $\geq$  10 alors
8     | retourner faux
9   | retourner vrai
10 cas où 8 faire
11   | retourner vrai
12 cas où 7 faire
13   | retourner ( $2 \leq$  valeurCroupier  $\leq$  7) cas où 6 faire
14     | retourner ( $2 \leq$  valeurCroupier  $\leq$  6) cas où 5 faire
15       | retourner faux
16     | cas où 4 faire
17       | retourner (valeurCroupier = 5 ou valeurCroupier = 6)
18     | cas où 3,2 faire
19       | retourner ( $2 \leq$  valeurCroupier  $\leq$  7)
20     | retourner faux
```

21

---

**Intégration dans le modèle**

Les deux stratégies implémentent l'interface **StrategieJoueur**, ce qui permet à la classe **JoueurRobot** de déléguer toutes ses décisions (tirer, rester, doubler, splitter, choisir la mise) sans connaître les détails internes des algorithmes. Ce découplage facilite l'ajout de nouvelles stratégies, la comparaison de leurs performances et les tests unitaires. Une même partie peut ainsi être jouée avec différents profils de robots, rendant l'application plus modulable et extensible.

# Chapitre 4

## Expérimentation

### 4.1 Lancement de l'application

Pour exécuter le projet, deux possibilités s'offrent à l'utilisateur :

- **Avec les commandes ant** : Le projet peut être lancé grâce aux différentes commandes prédéfinies dans le build.xml : **ant run**, **ant run-robot** (ou **run-robot-simple**), **ant run-robot-optimal**. Ces commandes ont également leur équivalent en passant par **ant run-custom -Dargs=""** avec les options **"robot simple"**, **"robot optimal"**, **"help"**
- **Après la distribution, avec l'exécution du jar** : Une fois le jar créé, le projet peut être lancé à l'aide de la commande **java -jar dist/blackjack-0.1.jar**. Des options telles que **robot**, **robot optimal**, **help** peuvent être ajoutées pour lancer la partie avec un robot dont on choisit la stratégie qu'il utilisera.

Une fois le projet exécuté, l'utilisateur est alors accueilli par l'écran d'accueil du jeu comme ci-dessous.



FIGURE 4.1 – Image d'accueil du jeu

## 4.2 L'interface en quelques images

Cette section présente l'interface graphique du jeu de Blackjack à travers différentes captures d'écran illustrant les principales étapes d'une partie.

### 4.2.1 Début de manche

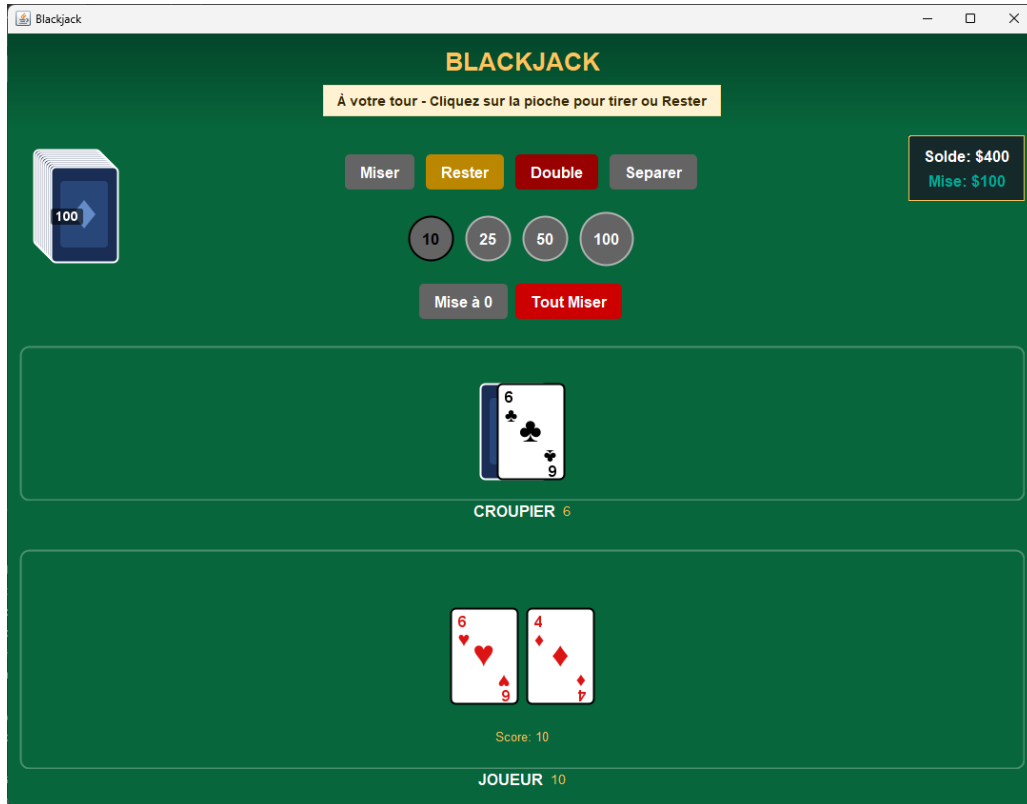


FIGURE 4.2 – Écran de début de manche

La figure 4.2 montre l'écran au début d'une manche. Le joueur dispose de plusieurs options pour placer sa mise : des boutons de montants prédéfinis (10, 25, 50, 100), ainsi que les actions classiques du Blackjack (Miser, Rester, Double, Séparer). Le croupier affiche sa première carte visible (6 de trèfle), tandis que le joueur reçoit ses deux cartes initiales (6 de cœur et 4 de carreau) pour un score de 10. Le solde du joueur (\$400) et sa mise actuelle (\$100) sont affichés dans le coin supérieur droit.



### 4.2.2 Partie en cours

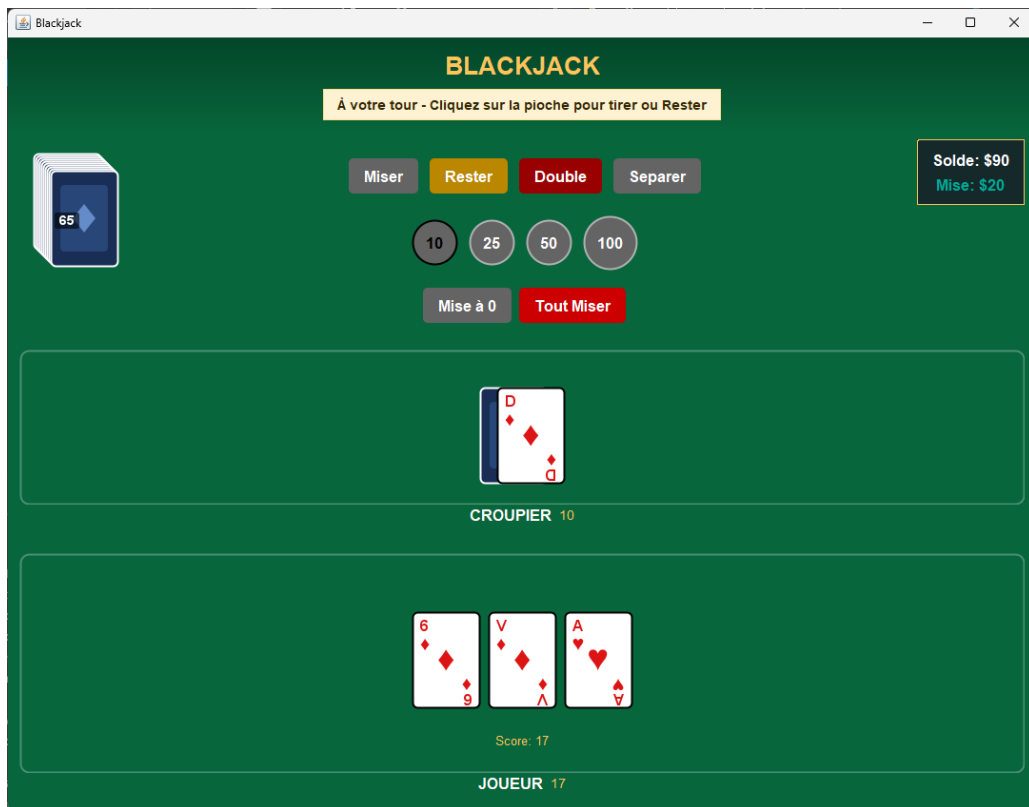


FIGURE 4.3 – Déroulement d'une manche

La figure 4.3 illustre une manche en cours de jeu. Le joueur a demandé une carte supplémentaire et possède maintenant trois cartes (6 de carreau, Valet de carreau, As de cœur) pour un score de 17. Le croupier affiche un 10 de carreau. Le bouton "Rester" est mis en évidence, indiquant que c'est au tour du joueur de décider s'il souhaite tirer une nouvelle carte ou rester sur son score actuel.

### 4.2.3 Mode avec robot

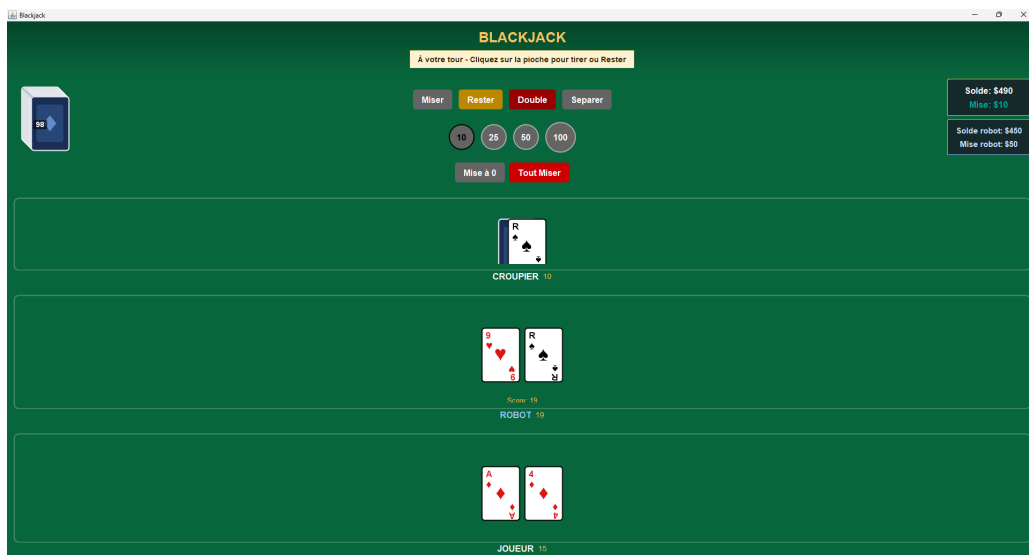


FIGURE 4.4 – Partie contre le robot

La figure 4.4 présente l'interface lorsque le mode robot est activé. En plus de la section du joueur, une section dédiée au robot apparaît, affichant ses cartes (9 de cœur et Roi de pique) et son score (19). Cette fonctionnalité permet de comparer les performances du joueur humain avec celles d'une intelligence artificielle. Les soldes respectifs du joueur et du robot sont visibles dans le coin supérieur droit (\$490 pour le joueur, \$450 pour le robot).

#### 4.2.4 Gestion du Split

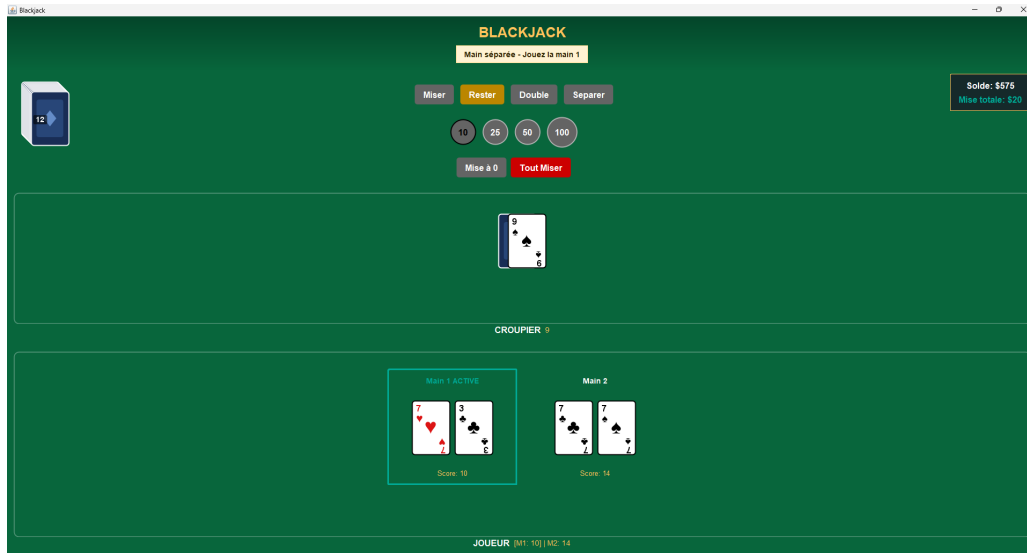


FIGURE 4.5 – Interface de séparation de paires (Split)

La figure 4.5 montre l'interface lorsque le joueur effectue un Split après avoir reçu une paire (7 de cœur et 7 de trèfle). Les deux mains sont clairement séparées : la "Main 1 ACTIVE" avec un 7 de cœur et un 3 de trèfle (score de 10) est en surbrillance cyan, tandis que la "Main 2" contient deux 7 de trèfle (score de 14). Le joueur peut jouer chaque main indépendamment, comme l'indique l'affichage au bas de l'écran "[M1 : 10] [M2 : 14]".

### 4.2.5 Victoire du joueur

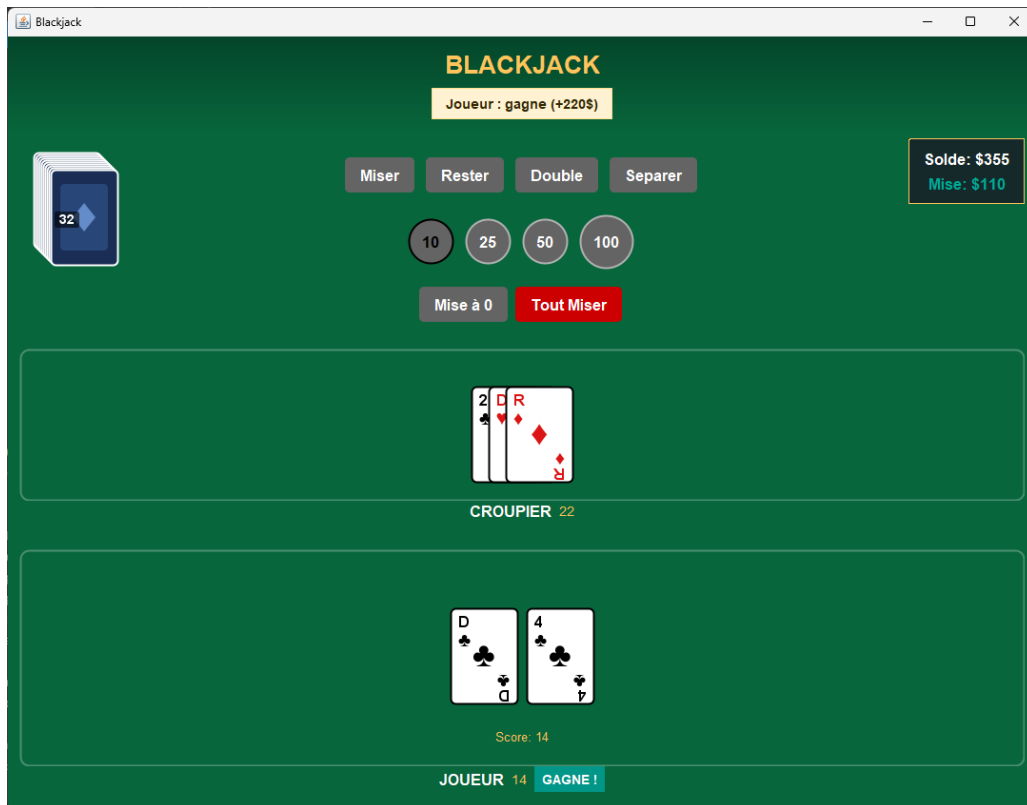


FIGURE 4.6 – Écran de victoire

La figure 4.6 illustre l'écran de fin de manche lorsque le joueur remporte la partie. Le message "Joueur : gagne (+220\$)" s'affiche en haut de l'écran. Le croupier a dépassé 21 points avec ses cartes (2 de pique, Dame de carreau, Roi de carreau pour un total de 22), tandis que le joueur a obtenu 14 points (Dame de trèfle et 4 de trèfle) et remporte donc la mise. Le solde du joueur est mis à jour en conséquence, passant à \$355.

# Conclusion

## 4.3 Conclusion

### 4.3.1 Bilan des travaux

Ce projet a abouti à la conception et au développement d'une application complète de Blackjack en Java, structurée selon une architecture modulaire à deux volets : une bibliothèque réutilisable de cartes (`cartes`) et un jeu de Blackjack (`blackjack`) exploitant cette bibliothèque. L'architecture repose sur le pattern MVC avec un système d'observation automatique entre les modèles (paquets de cartes) et leurs vues, garantissant une séparation claire des responsabilités et une maintenabilité optimale.

Le module `cartes` propose plusieurs vues spécialisées (`VuePaquetVisible`, `VuePaquetCache`, `VuePaquetEventail`) qui s'actualisent automatiquement via le pattern Observer lorsque les paquets sont modifiés. Le module `blackjack` implémente les règles complètes du jeu incluant le système de mises et de gestion de banque, le split de paires avec gestion multi-mains (classe `MainJoueur`), les paiements différenciés selon les résultats (blackjack naturel 3 :2, victoire standard 1 :1, push), et un système d'intelligence artificielle pour les joueurs robots avec deux stratégies configurables (`StrategieSimple` et `StrategieOptimale` basée sur la Basic Strategy mathématique du Blackjack).

### 4.3.2 Défis techniques surmontés

Le développement a nécessité de surmonter plusieurs obstacles techniques majeurs. La gestion des As avec leur valeur flexible (1 ou 11) a requis l'implémentation d'un algorithme optimal dans la classe `CalculateurScore` qui maximise le score sans dépasser 21. La mise en œuvre du split a exigé une refonte de l'architecture joueur pour supporter plusieurs mains simultanées, avec des règles spécifiques comme l'interdiction de tirer plus d'une carte sur des As splittés. Le système de paiement a été isolé dans une classe dédiée (`ServicePaiement`) qui gère tous les cas possibles (victoire, défaite, push, blackjack naturel) pour chaque main indépendamment, permettant ainsi de calculer correctement les gains même après un split.

### 4.3.3 Améliorations envisageables

Plusieurs axes d'amélioration peuvent être envisagés pour enrichir l'application. L'ajout de l'assurance lorsque le croupier montre un As constituerait une extension naturelle des règles actuelles. L'interface graphique bénéficierait d'animations pour la distribution des cartes et de sons d'ambiance pour améliorer l'expérience utilisateur.

Sur le plan de l'intelligence artificielle, bien que les stratégies actuelles (`StrategieSimple` et `StrategieOptimale`) offrent des comportements cohérents, elles pourraient être considérablement enrichies. Une IA avancée pourrait intégrer le comptage de cartes en suivant les cartes déjà jouées depuis le dernier reshuffle du sabot, ajuster dynamiquement les mises en fonction du *true count*, et implémenter des stratégies probabilistes prenant en compte la composition

exacte du sabot restant. L'ajout de profils de joueurs avec différents niveaux de risque (conservateur, équilibré, agressif) et l'implémentation d'algorithmes d'apprentissage permettraient à l'IA d'adapter sa stratégie en fonction de l'historique des parties et du comportement du joueur humain.

Le système de tests pourrait être complété avec une suite de tests unitaires JUnit couvrant les scénarios de jeu complexes, notamment les cas limites du split et du calcul de score avec plusieurs As. Enfin, l'ajout d'un historique des parties avec statistiques de gains/pertes et l'implémentation d'un système de sauvegarde permettraient aux joueurs de suivre leur progression sur plusieurs sessions.

L'architecture modulaire mise en place offre une base solide et extensible qui démontre une maîtrise approfondie des principes de conception orientée objet, des design patterns et du développement d'interfaces graphiques en Java.

# Bibliographie

- [1] CasinoUS. Basic strategy. [www.casinous.com/online-blackjack/basic-strategy/](http://www.casinous.com/online-blackjack/basic-strategy/). Accédé le 30 novembre 2025.